

NAME SINCHANA K
SRN PES1UG24AM405
CLASS G

Lab 3 – Written Justification

Problem Statement #07: Hostel Maintenance & Issue Ticketing System

Architecture Selection: “We chose Microservices Architecture for the Hostel Maintenance & Issue Ticketing System.”

Architectural Choice

We went with the Microservices style. The Portal UI, Auth & Access Control, Ticket Management, Attachment, Notification & SLA Escalation, and Reporting & Analytics components are each their own independently deployable service, talking to each other only through defined interfaces. We compared this against Layered and Client-Server before finalising it.

Reason 1 – the workloads are genuinely different from each other

Ticket creation happens constantly (every resident complaint), SLA escalation has to run continuously in the background checking timers, and Reporting only runs weekly/monthly when the Warden asks for it. These don't share a load pattern at all, so bundling them into one layered app means the whole app has to scale for the busiest one. As separate services we can scale Ticket Management during peak hostel-issue hours (like after a power cut) without touching Reporting.

Reason 2 – fault isolation actually matters here

If the Reporting & Analytics service crashes while generating a big monthly PDF, that shouldn't stop a resident from logging a burst pipe as an emergency ticket. In a tightly layered/monolithic setup a bug in one layer can bring the whole app down; with microservices, Ticket Management and Notification can keep running independently even if Reporting is having a bad day.

Security Advantage

Because Auth & Access Control is pulled out as its own service, every other service (Ticket Management, Reporting, etc.) has to call IAuthenticate rather than implement its own login/role check. That means there's exactly one place enforcing “Residents can't see Warden controls,” instead of that check being copy-pasted (and possibly forgotten) in multiple services.

Performance Benefit

Since Notification & SLA Escalation is a separate service, it can keep polling for 24-hour SLA breaches in the background without slowing down the actual ticket-submission API that residents are waiting on. Splitting them keeps the resident-facing path fast, which matters more for something urgent like plumbing/electrical issues.

Component Diagram

6 components inside the system boundary (Portal UI, Auth & Access Control, Ticket Management, Attachment, Notification & SLA Escalation, Reporting & Analytics) plus the external SMS/Email Gateway. 6 interfaces shown with ball (provided) / socket (required) notation.

